Part 4: Natural Language Processing (NLP)

Lino Galiana
Version of November 2, 2026

This part of the course introduces automatic language processing (NLP), a scientific field at the crossroads of linguistics and statistics that has become central to the field of *data science* as a result of the craze for generative AI. Using literary examples, this section first explores classic methods such as frequentist analysis and the processing of textual corpora in the form of *bag of words*. It then looks at language modelling, which opens the way to more original approaches. The aim of this chapter is to recall a few general points about the vast field of NLP

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **1**
2 Natural Language Processing **2**
3 Section Summary **2**
3.1 Text Cleaning and Frequency Analysis 2
3.2 Language Modeling 3
To Go Further **3**
Informations additionnelles 4

1 Introduction

The previous sections focused on acquiring cross-functional skills for working with data. Naturally, we have so far mostly focused on structured data—modest in size but already rich in analytical potential. This new section turns to a subject that, at first glance, may seem unlikely to be handled by computers—a topic of centuries-old philosophical debate, from Plato to Saussure: the richness of **human language**.

By drawing an analogy between “language” and “tongue”—that is, defining the former as the capacity to express and communicate thought through signs, and the latter as the conventional implementation of that capacity—we align ourselves with the field of linguistics and treat language as data. This opens the door to statistical and algorithmic analysis. Yet, even if statistical regularities exist, how can computers—ultimately limited to just *0* and *1*—grasp such a complex object as language, which takes humans years to understand and master?[1]

[1] While writing these lines, I asked `ChatGPT` to tell me a joke.

> Why do programmers hate the beach? Because the sand keeps changing paradigms!

Not understanding it, I asked for an explanation. Here’s what I got:

> Of course, I’d be happy to explain! This joke plays on the double meaning of the word “paradigm” and is a pun involving programming and the beach.

2 Natural Language Processing

Natural Language Processing (NLP) refers to the set of techniques that allow computers to understand, analyze, synthesize, and generate human language[2].

NLP is a disciplinary field at the intersection of statistics and linguistics, which has experienced significant growth in recent years — academically, operationally, and industrially. Some applications of these techniques have become essential in our daily lives, such as search engines, machine translation, and more recently, chatbots, whose development has accelerated rapidly since the launch of `ChatGPT` in December 2022.

3 Section Summary

This part of the course is dedicated to text data analysis with 📖 examples for fun. It serves as a gradual introduction to the topic by focusing on basic concepts necessary for later understanding of more advanced principles and sophisticated techniques[3]. This section mainly covers:

- The challenges of cleaning textual fields and frequency analysis. This is somewhat *old school* NLP but understanding it is essential to progress further;
- Language modeling using several approaches.

Before diving into the topic of *embeddings*, it's important to understand the contributions and limitations of concepts like the bag of words or TF-IDF (*term frequency - inverse document frequency*). One of the main benefits of large language models—namely the richness of their contextual window that allows them to better grasp textual nuances and speaker intentionality—becomes clearer when the limitations of traditional NLP are understood.

As an introductory perspective, this course focuses on frequency-based approaches, especially the bag-of-words approach, to ease into the later exploration of the Pandora's box that is *embeddings*.

3.1 Text Cleaning and Frequency Analysis

`Python` is an excellent tool for text data analysis. Basic methods for transforming textual data or dictionaries, combined with specialized libraries such as `NLTK` and `SpaCy`, make it possible to perform normalization and text data analysis very efficiently. `Python` is much better equipped than `R` for text data analysis. There is a wealth of online resources on this subject, and the best way to learn remains hands-on practice with a corpus to clean.

> In programming, a "paradigm" refers to a model or method of programming, such as object-oriented or functional programming. Programmers are often familiar with different paradigms.
>
> On the other hand, in a beach context, "paradigm" is humorously misused to mean "tide"—alluding to the natural shifting of sand and water.
>
> The joke plays on the humorous frustration programmers might feel when adapting to new programming paradigms, just like how sand constantly shifts with the tides. By combining both meanings of "paradigm," the joke creates an absurd and amusing scenario that will resonate with those familiar with programming and beaches.

In the end, I found the explanation funnier than the joke itself.

[2]The concept of "natural language processing" has tended to replace the more specific term "text mining." Text mining refers to the search for — or mining of — information within a set of texts. While it is one application area of NLP, it is not the only one. Advances in computing power and in the formalization of language have expanded the scope of fields where computational linguistics is applied.

[3]For example, the concept of *embedding*—the transformation of a text field into a multidimensional numeric vector —is central in NLP today but is only briefly mentioned here.

This section first revisits how to structure and clean a textual corpus through the *bag of words* approach. It aims to demonstrate how to turn a corpus into a tool suitable for statistical analysis:

- It first introduces the challenges of text data cleaning through an analysis of *The Count of Monte Cristo* by Alexandre Dumas here, which helps to quickly summarize the available information in a large volume of text data (as illustrated by wordclouds)
- It then offers a series of exercises on text cleaning based on the works of Edgar Allan Poe, Mary Shelley, and H.P. Lovecraft, aiming to highlight the specificity of each author's vocabulary. These exercises are available in the second chapter of the section.

This frequency-based analysis provides perspective on the nature of text data and recurring issues in dimensionality reduction of natural language corpora. Just as descriptive statistics naturally lead to modeling, this frequency approach typically quickly leads to the desire to identify underlying rules behind our text corpora.

3.2 Language Modeling

The remainder of this section introduces the challenges of language modeling. These are currently very popular due to the success of `ChatGPT`. However, before delving into large language models (LLMs)—those neural networks with billions of parameters trained on massive data volumes—it's important to first understand some preliminary modeling techniques.

We begin by exploring an alternative approach that takes into account the context in which a word appears. The introduction of *Latent Dirichlet Allocation* (LDA) serves as an opportunity to present document modeling through *topics*. However, this approach has fallen out of favor in comparison to methods related to the concept of *embedding*.

Toward the end of this course section, we will introduce the challenge of transforming textual fields into numeric vector forms. To do so, we will present the principle behind `Word2Vec`, which allows us, for instance, despite significant syntactic distance, to identify that semantically, `Man` and `Woman` are closely related. This chapter serves as a bridge to the concept of *embedding*, a major recent revolution in NLP. It enables the comparison of corpora not only by syntactic similarity (e.g., do they share common words?) but also by semantic similarity (e.g., do they share a theme or meaning?). Covering `Word2Vec` will give curious learners a solid foundation to then explore transformer-based models, which are now the benchmark in NLP.

To Go Further

Research in the field of NLP is highly active. It is therefore advisable to stay curious and explore additional resources, as no single source can compile all knowledge—especially in a field as dynamic as NLP.

To deepen the skills discussed in this course, I strongly recommend this course by `HuggingFace`.

To understand the internal architecture of an LLM, this post by Sebastian Raschka is very helpful.

These chapters only scratch the surface of NLP use cases for *data scientists*. For instance, in public statistics, one major NLP use case involves using automatic classification techniques to convert free-text answers in surveys into predefined fields within a nomenclature. This is a specific adaptation to public statistics, a heavy user of standardized nomenclatures, of multi-level classification problems.

Here is an example from a project on automated job classification using the PCS (socio-professional categories) typology, based on a model trained with the `Fasttext` library:

```
import requests
import pandas as pd

activite = "data scientist"
urlApe = (
    "https://codification-ape-test.lab.sspcloud.fr/"
    f"predict?nb_echos_max=3&prob_min=0&text_feature={activite}"
)

try:
    # requête
    resp = requests.get(urlApe, timeout=10)
    resp.raise_for_status()  # lève une erreur si code HTTP ≠ 200
    data = resp.json()

    # récupération de IC
    IC = data.pop("IC", None)

    # transformation en DataFrame
    df = pd.DataFrame(data.values())
    df["indice_confiance"] = IC

    print(df)

except requests.exceptions.RequestException as e:
    print("Erreur lors de l'appel API :", e)
    df = pd.DataFrame()  # DataFrame vide en cas d'échec

except (ValueError, KeyError) as e:
    print("Erreur lors du parsing des données :", e)
    df = pd.DataFrame()
```

```
Erreur lors de l'appel API : 503 Server Error: Service Unavailable for url: https://codification-ape-test.lab.sspcloud.fr/predict?nb_echos_max=3&prob_min=0&text_feature=data%20scientist
```

Informations additionnelles

i Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
```

```
    "contextily==1.6.2",
    "duckdb>=0.10.1",
    "folium>=0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables>=0.12.0",
    "gt-extras>=0.0.8",
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
```

```
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.